Surat Technical Education & Research Society's
Smt. Kalavatiben Fulchandbhai Vakhariya Education Complex
SASCMA EMCC & H.H. BBA &
U.J.B. BCA & Msc(IT) & STERS BA
(Psychology) & B.sc. (Data Science)
Affiliated to Veer Narmad South Gujarat University

FYBSC(IT) [SEM-2]

Practical Journal

Year : 2025-26

Subject Code: 205

Subject Name: Practical – II(DBMS)

Roll No.: 1014

Student Name: Prince Sureshbhai Kahar

Class : FYBSC(IT) [SEM-2]

Div: A

Faculty Name: Dr. Hemang Desai

Faculty Sign.: _____

Date: 06 / 03 / 2026

DBMS [Assignment – 1]

Table Name: Students		
Sr. No	**Field Name**	**Data Type**
1	StudentID	INTEGER
2	Name	VARCHAR(100)
3	Age	INTEGER
4	Gender	VARCHAR(6)
5	City	VARCHAR(100)
6	Marks	INTEGER

Query 1:	Create the above table using SQL command.
Answer:	CREATE TABLE Students(StudentID INT PRIMARY KEY, Name VARCHAR (100), Age INT, GENDER VARCHAR(6), CITY VARCHAR (100), Marks INT);
Query 2:	Insert **Ten records** in above table using SQL Commands.
Answer:	INSERT INTO Students VALUES (1, 'Prince', 19, 'Male', 'Surat', 80), (2, 'Priyaa', 21, 'Female', 'Mumbai', 70), (3, 'Mahir', 21, 'Male', 'Surat', 87), (4, 'Shaily', 19, 'Female', 'Surat', 60), (5, 'Suresh', 22, 'Male', 'Ahmedabad', 76), (6, 'Dhruhi', 18, 'Female', 'Mumbai', 83), (7, 'Himesh', 20, 'Male', 'Surat', 65), (8, 'Yashvi', 19, 'Female', 'Bangalore', 90), (9, 'Aniket', 23, 'Male', 'Pune', 73), (10, 'Zeel', 21, 'Female', 'Surat', 77) ;
OUTPUT:	

StudentID	Name	Age	GENDER	CITY	Marks
1	Prince	19	Male	Surat	80
2	Priyaa	21	Female	Mumbai	70
3	Mahir	21	Male	Surat	87
4	Shaily	19	Female	Surat	60
5	Suresh	22	Male	Ahmedabad	76
6	Dhruhi	18	Female	Mumbai	83
7	Himesh	20	Male	Surat	65
8	Yashvi	19	Female	Bangalore	90
9	Aniket	23	Male	Pune	73
10	Zeel	21	Female	Surat	77

Query 3:	Display the Name and City of all students.
Answer:	SELECT Name, City FROM Students;

OUTPUT:

Name	City
Prince	Surat
Priyaa	Mumbai
Mahir	Surat
Shaily	Surat
Suresh	Ahmedabad
Dhruhi	Mumbai
Himesh	Surat
Yashvi	Bangalore
Aniket	Pune
Zeel	Surat

Query 4:	Find the details of students whose City is 'Surat'.
Answer:	SELECT * FROM Students WHERE City = 'Surat';

OUTPUT:

StudentID	Name	Age	GENDER	CITY	Marks
1	Prince	19	Male	Surat	80
3	Mahir	21	Male	Surat	87
4	Shaily	19	Female	Surat	60
7	Himesh	20	Male	Surat	65
10	Zeel	21	Female	Surat	77

Query 5:	Retrieve the names of all female students.
Answer:	SELECT Name FROM Students WHERE Gender = 'Female';

OUTPUT:

Name
Priyaa
Shaily
Dhruhi
Yashvi
Zeel

Query 6:	Find the students who have scored more than 80 marks.
Answer:	SELECT * FROM Students WHERE Marks > 80;

OUTPUT:

StudentID	Name	Age	GENDER	CITY	Marks
3	Mahir	21	Male	Surat	87
6	Dhruhi	18	Female	Mumbai	83
8	Yashvi	19	Female	Bangalore	90

Query 7:	Find the details of students aged 20.
Answer:	SELECT * FROM Students WHERE Age = 20;

OUTPUT:

StudentID	Name	Age	GENDER	CITY	Marks
7	Himesh	20	Male	Surat	65

Query 8:	Retrieve records of students whose Marks are between 70 and 90.
Answer:	SELECT * FROM Students WHERE Marks BETWEEN 70 AND 90;

OUTPUT:

StudentID	Name	Age	GENDER	CITY	Marks
1	Prince	19	Male	Surat	80
2	Priyaa	21	Female	Mumbai	70
3	Mahir	21	Male	Surat	87
5	Suresh	22	Male	Ahmedabad	76
6	Dhruhi	18	Female	Mumbai	83
8	Yashvi	19	Female	Bangalore	90
9	Aniket	23	Male	Pune	73
10	Zeel	21	Female	Surat	77

Query 9:	Display the names of students not from 'Mumbai'.
Answer:	SELECT Name FROM Students WHERE City != 'Mumbai';

OUTPUT:

```
+--------+
| Name   |
+--------+
| Prince |
| Mahir  |
| Shaily |
| Suresh |
| Himesh |
| Yashvi |
| Aniket |
| Zeel   |
+--------+
```

Query 10:	Find students whose names start with the letter 'A'.
Answer:	SELECT * FROM Students WHERE Name LIKE 'A%';

OUTPUT:

```
+-----------+--------+------+--------+------+-------+
| StudentID | Name   | Age  | GENDER | CITY | Marks |
+-----------+--------+------+--------+------+-------+
|         9 | Aniket |   23 | Male   | Pune |    73 |
+-----------+--------+------+--------+------+-------+
```

Query 11:	Retrieve records of male students aged above 21.
Answer:	SELECT * FROM Students WHERE Gender = 'Male' AND Age > 21;

OUTPUT:

StudentID	Name	Age	GENDER	CITY	Marks
5	Suresh	22	Male	Ahmedabad	76
9	Aniket	23	Male	Pune	73

Query 12:	Display all student records sorted by Marks in ascending order.
Answer:	SELECT * FROM Students ORDER BY Marks ASC;

OUTPUT:

StudentID	Name	Age	GENDER	CITY	Marks
4	Shaily	19	Female	Surat	60
7	Himesh	20	Male	Surat	65
2	Priyaa	21	Female	Mumbai	70
9	Aniket	23	Male	Pune	73
5	Suresh	22	Male	Ahmedabad	76
10	Zeel	21	Female	Surat	77
1	Prince	19	Male	Surat	80
6	Dhruhi	18	Female	Mumbai	83
3	Mahir	21	Male	Surat	87
8	Yashvi	19	Female	Bangalore	90

DBMS [Assignment – 2]

Table Name: Courses		
Sr. No	**Field Name**	**Data Type**
1	CourseID	INTEGER
2	CourseName	VARCHAR(100)
3	Duration	VARCHAR(20)
4	Fees	INTEGER

Query 1:	Create the above table using SQL command.
Answer:	CREATE TABLE Courses (CourseID INT PRIMARY KEY, CourseName VARCHAR(50), Duration VARCHAR(20), Fees INT);
Query 2:	Insert **Ten records** in above table using SQL Commands.
Answer:	INSERT INTO Courses (CourseID, CourseName, Duration, Fees) VALUES (101, 'Data Science', '6 months', 25000), (102, 'Data Analytics', '4 months', 18000), (103, 'Python Basics', '2 months', 8000), (104, 'Java Programming', '6 months', 22000), (105, 'Web Design', '3 months', 12000), (106, 'Cloud Computing', '6 months', 28000), (107, 'UI/UX Design', '3 months', 15000), (108, 'Big Data Engineer', '8 months', 35000), (109, 'Cyber Security', '5 months', 30000), (110, 'Digital Marketing', '2 months', 10000) ;
OUTPUT:	

CourseID	CourseName	Duration	Fees
101	Data Science	6 months	25000
102	Data Analytics	4 months	18000
103	Python Basics	2 months	8000
104	Java Programming	6 months	22000
105	Web Design	3 months	12000
106	Cloud Computing	6 months	28000
107	UI/UX Design	3 months	15000
108	Big Data Engineer	8 months	35000
109	Cyber Security	5 months	30000
110	Digital Marketing	2 months	10000

Query 3:	Display the CourseName and Fees of all courses.
Answer:	SELECT CourseName, Fees FROM Courses;

OUTPUT:

CourseName	Fees
Data Science	25000
Data Analytics	18000
Python Basics	8000
Java Programming	22000
Web Design	12000
Cloud Computing	28000
UI/UX Design	15000
Big Data Engineer	35000
Cyber Security	30000
Digital Marketing	10000

Query 4:	Find the details of courses with a duration of '6 months'.

Answer:	SELECT * FROM Courses WHERE Duration = '6 months';

OUTPUT:

```
+----------+------------------+----------+-------+
| CourseID | CourseName       | Duration | Fees  |
+----------+------------------+----------+-------+
|      101 | Data Science     | 6 months | 25000 |
|      104 | Java Programming | 6 months | 22000 |
|      106 | Cloud Computing  | 6 months | 28000 |
+----------+------------------+----------+-------+
```

Query 5:	Retrieve the names of courses with fees greater than 15000.
Answer:	SELECT CourseName FROM Courses WHERE Fees > 15000;

OUTPUT:

```
+--------------------+
| CourseName         |
+--------------------+
| Data Science       |
| Data Analytics     |
| Java Programming   |
| Cloud Computing    |
| Big Data Engineer  |
| Cyber Security     |
+--------------------+
```

Query 6:	Find the course with the highest fees.
Answer:	SELECT * FROM Courses WHERE Fees = (SELECT MAX(Fees) FROM Courses);

OUTPUT:

```
+----------+--------------------+----------+-------+
| CourseID | CourseName         | Duration | Fees  |
+----------+--------------------+----------+-------+
|      108 | Big Data Engineer  | 8 months | 35000 |
+----------+--------------------+----------+-------+
```

Query 7:	Count the total number of courses available.
Answer:	SELECT COUNT(*) AS TotalCourses FROM Courses;

OUTPUT:

```
+--------------+
| TotalCourses |
+--------------+
|           10 |
+--------------+
```

Query 8:	Find the average fees of all courses.
Answer:	SELECT AVG(Fees) AS AverageFees FROM Courses;

OUTPUT:

```
+--------------+
| AverageFees  |
+--------------+
|  20300.0000  |
+--------------+
```

Query 9:	Retrieve courses where the CourseName contains the word 'Data'.
Answer:	SELECT * FROM Courses WHERE CourseName LIKE '%Data%';

OUTPUT:

CourseID	CourseName	Duration	Fees
101	Data Science	6 months	25000
102	Data Analytics	4 months	18000
108	Big Data Engineer	8 months	35000

Query 10:	Display the course names and fees where the fees are less than or equal to 20000.
Answer:	SELECT CourseName, Fees FROM Courses WHERE Fees <= 20000;

OUTPUT;

CourseName	Fees
Data Analytics	18000
Python Basics	8000
Web Design	12000
UI/UX Design	15000
Digital Marketing	10000

Query 11:	Find the course with the shortest duration.
Answer:	SELECT * FROM Courses ORDER BY Duration ASC LIMIT 1;

OUTPUT:

CourseID	CourseName	Duration	Fees
103	Python Basics	2 months	8000

Query 12:	Retrieve course names with a duration ending in 'months'.

Answer:	SELECT CourseName FROM Courses WHERE Duration LIKE '%months';

OUTPUT:

```
+--------------------+
| CourseName         |
+--------------------+
| Data Science       |
| Data Analytics     |
| Python Basics      |
| Java Programming   |
| Web Design         |
| Cloud Computing    |
| UI/UX Design       |
| Big Data Engineer  |
| Cyber Security     |
| Digital Marketing  |
+--------------------+
```

DBMS [Assignment – 3]

Table Name: library		
Sr. No	**Field Name**	**Data Type**
1	book_id	INTEGER
2	book_name	VARCHAR(100)
3	author	VARCHAR(50)
4	category	VARCHAR(50)
5	price	DECIMAL(10,2)
6	quantity	INTEGER

Query 1:	Create a database named library_db.
Answer:	CREATE DATABASE library_db; USE library_db;
Query 2:	Create the above table and Set book_id as Primary Key.
Answer:	CREATE TABLE library (book_id INT PRIMARY KEY, book_name VARCHAR(100), author VARCHAR(100), category VARCHAR(50), price DECIMAL(10, 2), quantity INT);
Query 3:	Display the structure of the library table.
Answer:	DESC library;
OUTPUT:	

```
+-----------+---------------+------+-----+---------+-------+
| Field     | Type          | Null | Key | Default | Extra |
+-----------+---------------+------+-----+---------+-------+
| book_id   | int           | NO   | PRI | NULL    |       |
| book_name | varchar(100)  | YES  |     | NULL    |       |
| author    | varchar(100)  | YES  |     | NULL    |       |
| category  | varchar(50)   | YES  |     | NULL    |       |
| price     | decimal(10,2) | YES  |     | NULL    |       |
| quantity  | int           | YES  |     | NULL    |       |
+-----------+---------------+------+-----+---------+-------+
```

Query 4:	Insert at least 5 book records.
Answer:	INSERT INTO library VALUES (101, 'Data Structures', 'Lipschutz', 'Computer', 450.00, 15), (102, 'Digital Logic', 'Morris Mano', 'Electronics', 600.00, 8), (103, 'Python Programming', 'Reema Thareja', 'Computer', 550.00, 20), (104, 'Calculus', 'Stewart', 'Math', 750.00, 5), (105, 'Database Systems', 'Korth', 'Computer', 800.00, 12) ;

OUTPUT:

```
+---------+--------------------+---------------+-------------+--------+----------+
| book_id | book_name          | author        | category    | price  | quantity |
+---------+--------------------+---------------+-------------+--------+----------+
|     101 | Data Structures    | Lipschutz     | Computer    | 450.00 |       15 |
|     102 | Digital Logic      | Morris Mano   | Electronics | 600.00 |        8 |
|     103 | Python Programming | Reema Thareja | Computer    | 550.00 |       20 |
|     104 | Calculus           | Stewart       | Math        | 750.00 |        5 |
|     105 | Database Systems   | Korth         | Computer    | 800.00 |       12 |
+---------+--------------------+---------------+-------------+--------+----------+
```

Query 5:	Insert one record by specifying column names.
Answer:	INSERT INTO library (book_id, book_name, author, category, price, quantity) VALUES (106, 'Physics Concepts', 'H.C. Verma', 'Science', 400.00, 25) ;

OUTPUT:

book_id	book_name	author	category	price	quantity
101	Data Structures	Lipschutz	Computer	450.00	15
102	Digital Logic	Morris Mano	Electronics	600.00	8
103	Python Programming	Reema Thareja	Computer	550.00	20
104	Calculus	Stewart	Math	750.00	5
105	Database Systems	Korth	Computer	800.00	12
106	Physics Concepts	H.C. Verma	Science	400.00	25

Query 6:	Display all book records.
Answer:	SELECT * FROM library;

OUTPUT:

book_id	book_name	author	category	price	quantity
101	Data Structures	Lipschutz	Computer	450.00	15
102	Digital Logic	Morris Mano	Electronics	600.00	8
103	Python Programming	Reema Thareja	Computer	550.00	20
104	Calculus	Stewart	Math	750.00	5
105	Database Systems	Korth	Computer	800.00	12
106	Physics Concepts	H.C. Verma	Science	400.00	25

Query 7:	Display book_name and author only.
Answer:	SELECT book_name, author FROM library;

OUTPUT:

book_name	author
Data Structures	Lipschutz
Digital Logic	Morris Mano
Python Programming	Reema Thareja
Calculus	Stewart
Database Systems	Korth
Physics Concepts	H.C. Verma

Query 8:	Display books belonging to 'Computer' category.
Answer:	SELECT * FROM library WHERE category = 'Computer';

OUTPUT:

book_id	book_name	author	category	price	quantity
101	Data Structures	Lipschutz	Computer	450.00	15
103	Python Programming	Reema Thareja	Computer	550.00	20
105	Database Systems	Korth	Computer	800.00	12

Query 9:	Display books with price greater than 500.
Answer:	SELECT * FROM library WHERE price > 500;

OUTPUT:

book_id	book_name	author	category	price	quantity
102	Digital Logic	Morris Mano	Electronics	600.00	8
103	Python Programming	Reema Thareja	Computer	550.00	20
104	Calculus	Stewart	Math	750.00	5
105	Database Systems	Korth	Computer	800.00	12

Query 10:	Display books ordered by price (ascending).
Answer:	SELECT * FROM library ORDER BY price;

OUTPUT:

book_id	book_name	author	category	price	quantity
106	Physics Concepts	H.C. Verma	Science	400.00	25
101	Data Structures	Lipschutz	Computer	450.00	15
103	Python Programming	Reema Thareja	Computer	550.00	20
102	Digital Logic	Morris Mano	Electronics	600.00	8
104	Calculus	Stewart	Math	750.00	5
105	Database Systems	Korth	Computer	800.00	12

Query 11:	Display books whose quantity is less than 10.
Answer:	SELECT * FROM library WHERE quantity < 10;

OUTPUT;

book_id	book_name	author	category	price	quantity
102	Digital Logic	Morris Mano	Electronics	600.00	8
104	Calculus	Stewart	Math	750.00	5

Query 12:	Update price of a book using book_id.
Answer:	UPDATE library SET price = 480.00 WHERE book_id = 101;

OUTPUT:

book_id	book_name	author	category	price	quantity
101	Data Structures	Lipschutz	Computer	480.00	15
102	Digital Logic	Morris Mano	Electronics	600.00	8
103	Python Programming	Reema Thareja	Computer	550.00	20
104	Calculus	Stewart	Math	750.00	5
105	Database Systems	Korth	Computer	800.00	12
106	Physics Concepts	H.C. Verma	Science	400.00	25

Query 13:	Increase the price of all books by 10%.

Answer:	UPDATE library SET price = price * 1.10;

OUTPUT:

book_id	book_name	author	category	price	quantity
101	Data Structures	Lipschutz	Computer	528.00	15
102	Digital Logic	Morris Mano	Electronics	660.00	8
103	Python Programming	Reema Thareja	Computer	605.00	20
104	Calculus	Stewart	Math	825.00	5
105	Database Systems	Korth	Computer	880.00	12
106	Physics Concepts	H.C. Verma	Science	440.00	25

Query 14:	Delete a book whose book_id is given.
Answer:	DELETE FROM library WHERE book_id = 104;

OUTPUT:

book_id	book_name	author	category	price	quantity
101	Data Structures	Lipschutz	Computer	528.00	15
102	Digital Logic	Morris Mano	Electronics	660.00	8
103	Python Programming	Reema Thareja	Computer	605.00	20
105	Database Systems	Korth	Computer	880.00	12
106	Physics Concepts	H.C. Verma	Science	440.00	25

Query 15:	Count total books.
Answer:	SELECT COUNT(*) AS TotalBooks FROM library;

OUTPUT:

TotalBooks
5

Query 16:	Find the maximum book price.
Answer:	SELECT MAX(price) AS MaximumPrice FROM library;

OUTPUT:

```
+--------------+
| MaximumPrice |
+--------------+
|       880.00 |
+--------------+
```

Query 17:	Find the average book price.
Answer:	SELECT AVG(price) AS AveragePrice FROM library;

OUTPUT:

```
+--------------+
| AveragePrice |
+--------------+
|   622.600000 |
+--------------+
```

DBMS [Assignment – 4]

Table Name: product		
Sr. No	**Field Name**	**Data Type**
1	product_id	INTEGER
2	product_name	VARCHAR(100)
3	brand	VARCHAR(50)
4	category	VARCHAR(50)
5	price	DECIMAL(10,2)
6	stock	INTEGER

Query 1:	Create a database named shop_db.
Answer:	CREATE DATABASE shop_db; USE shop_db;
Query 2:	Create the above table and Set product_id as Primary Key.
Answer:	CREATE TABLE product (product_id INT PRIMARY KEY, product_name VARCHAR(100), brand VARCHAR(50), category VARCHAR(50), price DECIMAL(10, 2), stock INT);
Query 3:	Display the structure of the product table.
Answer:	DESC product;
OUTPUT:	

```
+--------------+---------------+------+-----+---------+-------+
| Field        | Type          | Null | Key | Default | Extra |
+--------------+---------------+------+-----+---------+-------+
| product_id   | int           | NO   | PRI | NULL    |       |
| product_name | varchar(100)  | YES  |     | NULL    |       |
| brand        | varchar(50)   | YES  |     | NULL    |       |
| category     | varchar(50)   | YES  |     | NULL    |       |
| price        | decimal(10,2) | YES  |     | NULL    |       |
| stock        | int           | YES  |     | NULL    |       |
+--------------+---------------+------+-----+---------+-------+
```

Query 4:	Insert at least 5 product records.
Answer:	INSERT INTO product VALUES (1, 'Smartphone', 'Samsung', 'Electronics', 45000.00, 25), (2, 'Laptop', 'HP', 'Electronics', 65000.00, 15), (3, 'Bluetooth Speaker', 'JBL', 'Electronics', 5000.00, 50), (4, 'Microwave', 'LG', 'Appliances', 12000.00, 10), (5, 'Headphones', 'Sony', 'Electronics', 15000.00, 18) ;

OUTPUT:

```
+------------+--------------------+---------+-------------+----------+-------+
| product_id | product_name       | brand   | category    | price    | stock |
+------------+--------------------+---------+-------------+----------+-------+
|          1 | Smartphone         | Samsung | Electronics | 45000.00 |    25 |
|          2 | Laptop             | HP      | Electronics | 65000.00 |    15 |
|          3 | Bluetooth Speaker  | JBL     | Electronics |  5000.00 |    50 |
|          4 | Microwave          | LG      | Appliances  | 12000.00 |    10 |
|          5 | Headphones         | Sony    | Electronics | 15000.00 |    18 |
+------------+--------------------+---------+-------------+----------+-------+
```

Query 5:	Insert one product record using column names.
Answer:	INSERT INTO product (product_id, product_name, brand, category, price, stock) VALUES (6, 'Desk Lamp', 'Philips', 'Lighting', 1500.00, 30) ;

OUTPUT:

product_id	product_name	brand	category	price	stock
1	Smartphone	Samsung	Electronics	45000.00	25
2	Laptop	HP	Electronics	65000.00	15
3	Bluetooth Speaker	JBL	Electronics	5000.00	50
4	Microwave	LG	Appliances	12000.00	10
5	Headphones	Sony	Electronics	15000.00	18
6	Desk Lamp	Philips	Lighting	1500.00	30

Query 6:	Display all product records.
Answer:	SELECT * FROM product;

OUTPUT:

product_id	product_name	brand	category	price	stock
1	Smartphone	Samsung	Electronics	45000.00	25
2	Laptop	HP	Electronics	65000.00	15
3	Bluetooth Speaker	JBL	Electronics	5000.00	50
4	Microwave	LG	Appliances	12000.00	10
5	Headphones	Sony	Electronics	15000.00	18
6	Desk Lamp	Philips	Lighting	1500.00	30

Query 7:	Display product_name and price only.
Answer:	SELECT product_name, price FROM product;

OUTPUT:

product_name	price
Smartphone	45000.00
Laptop	65000.00
Bluetooth Speaker	5000.00
Microwave	12000.00
Headphones	15000.00
Desk Lamp	1500.00

Query 8:	Display products of 'Electronics' category.
Answer:	SELECT * FROM product WHERE category = 'Electronics';

OUTPUT:

product_id	product_name	brand	category	price	stock
1	Smartphone	Samsung	Electronics	45000.00	25
2	Laptop	HP	Electronics	65000.00	15
3	Bluetooth Speaker	JBL	Electronics	5000.00	50
5	Headphones	Sony	Electronics	15000.00	18

Query 9:	Display products with prices between 10000 and 50000.
Answer:	SELECT * FROM product WHERE price BETWEEN 10000 AND 50000;

OUTPUT:

product_id	product_name	brand	category	price	stock
1	Smartphone	Samsung	Electronics	45000.00	25
4	Microwave	LG	Appliances	12000.00	10
5	Headphones	Sony	Electronics	15000.00	18

Query 10:	Display products ordered by price (descending).
Answer:	SELECT * FROM product ORDER BY price DESC;

OUTPUT:

product_id	product_name	brand	category	price	stock
2	Laptop	HP	Electronics	65000.00	15
1	Smartphone	Samsung	Electronics	45000.00	25
5	Headphones	Sony	Electronics	15000.00	18
4	Microwave	LG	Appliances	12000.00	10
3	Bluetooth Speaker	JBL	Electronics	5000.00	50
6	Desk Lamp	Philips	Lighting	1500.00	30

Query 11:	Display products whose stock is less than 20.
Answer:	SELECT * FROM product WHERE stock < 20;

OUTPUT;

product_id	product_name	brand	category	price	stock
2	Laptop	HP	Electronics	65000.00	15
4	Microwave	LG	Appliances	12000.00	10
5	Headphones	Sony	Electronics	15000.00	18

Query 12:	Update stock of a product using product_id.
Answer:	UPDATE product SET stock = 45 WHERE product_id = 3;

OUTPUT:

product_id	product_name	brand	category	price	stock
1	Smartphone	Samsung	Electronics	45000.00	25
2	Laptop	HP	Electronics	65000.00	15
3	Bluetooth Speaker	JBL	Electronics	5000.00	45
4	Microwave	LG	Appliances	12000.00	10
5	Headphones	Sony	Electronics	15000.00	18
6	Desk Lamp	Philips	Lighting	1500.00	30

Query 13:	Increase the price of all products by 5%.
Answer:	UPDATE product SET price = price * 1.05;

OUTPUT:

product_id	product_name	brand	category	price	stock
1	Smartphone	Samsung	Electronics	47250.00	25
2	Laptop	HP	Electronics	68250.00	15
3	Bluetooth Speaker	JBL	Electronics	5250.00	45
4	Microwave	LG	Appliances	12600.00	10
5	Headphones	Sony	Electronics	15750.00	18
6	Desk Lamp	Philips	Lighting	1575.00	30

Query 14:	Delete a product whose product_id is given.
Answer:	DELETE FROM product WHERE product_id = 6;

OUTPUT:

product_id	product_name	brand	category	price	stock
1	Smartphone	Samsung	Electronics	47250.00	25
2	Laptop	HP	Electronics	68250.00	15
3	Bluetooth Speaker	JBL	Electronics	5250.00	45
4	Microwave	LG	Appliances	12600.00	10
5	Headphones	Sony	Electronics	15750.00	18

Query 15:	Count total products.
Answer:	SELECT COUNT(*) AS TotalProducts FROM product;

OUTPUT:

```
+---------------+
| TotalProducts |
+---------------+
|             5 |
+---------------+
```

Query 16:	Find the highest product price.
Answer:	SELECT MAX(price) AS HighestPrice FROM product;

OUTPUT:

```
+--------------+
| HighestPrice |
+--------------+
|     68250.00 |
+--------------+
```

Query 17:	Find average product price.
Answer:	SELECT AVG(price) AS AveragePrice FROM product;

OUTPUT:

```
+--------------+
| AveragePrice |
+--------------+
| 29820.000000 |
+--------------+
```

DBMS [Assignment – 5]

Table Name: patient		
Sr. No	**Field Name**	**Data Type**
1	patient_id	INTEGER
2	patient_name	VARCHAR(100)
3	gender	VARCHAR(6)
4	age	INTEGER
5	disease	VARCHAR(100)
6	admission_date	DATE
7	bill_amount	DECIM AL(10,2)

Query 1:	Create a database named hospital_db.
Answer:	CREATE DATABASE hospital_db; USE hospital_db;
Query 2:	Create the above table and Set patient_id as Primary Key.
Answer:	CREATE TABLE patient (patient_id INT PRIMARY KEY, patient_name VARCHAR(100), gender CHAR(6), age INT, disease VARCHAR(100), admission_date DATE, bill_amount DECIMAL(10, 2));
Query 3:	Display the structure of the patient table.
Answer:	DESC patient;
OUTPUT:	

```
+----------------+---------------+------+-----+---------+-------+
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| patient_id     | int           | NO   | PRI | NULL    |       |
| patient_name   | varchar(100)  | YES  |     | NULL    |       |
| gender         | char(6)       | YES  |     | NULL    |       |
| age            | int           | YES  |     | NULL    |       |
| disease        | varchar(100)  | YES  |     | NULL    |       |
| admission_date | date          | YES  |     | NULL    |       |
| bill_amount    | decimal(10,2) | YES  |     | NULL    |       |
+----------------+---------------+------+-----+---------+-------+
```

Query 4:	Insert at least 5 patient records.
Answer:	INSERT INTO patient VALUES (501, 'Himanshu', 'Male', 65, 'Fever', '2023-10-01', 5000.00), (502, 'Shaily', 'Female', 28, 'Malaria', '2023-10-05', 25000.00), (503, 'Het', 'Male', 72, 'Fever', '2023-10-08', 12000.00), (504, 'Yashvi', 'Female', 45, 'Flu', '2023-10-10', 30000.00), (505, 'Karan', 'Male', 15, 'Injured Leg', '2023-10-12', 15000.00) ;

OUTPUT:

```
+------------+--------------+--------+------+-------------+----------------+-------------+
| patient_id | patient_name | gender | age  | disease     | admission_date | bill_amount |
+------------+--------------+--------+------+-------------+----------------+-------------+
|        501 | Himanshu     | Male   |   65 | Fever       | 2023-10-01     |     5000.00 |
|        502 | Shaily       | Female |   28 | Malaria     | 2023-10-05     |    25000.00 |
|        503 | Het          | Male   |   72 | Fever       | 2023-10-08     |    12000.00 |
|        504 | Yashvi       | Female |   45 | Flu         | 2023-10-10     |    30000.00 |
|        505 | Karan        | Male   |   15 | Injured Leg | 2023-10-12     |    15000.00 |
+------------+--------------+--------+------+-------------+----------------+-------------+
```

Query 5:	Insert one patient record using column names.
Answer:	INSERT INTO patient (patient_id, patient_name, gender, age, disease, admission_date, bill_amount) VALUES (506, 'Dhruhi', 'Female', 30, 'Fever', '2023-10-15', 8000.00);

OUTPUT:

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
501	Himanshu	Male	65	Fever	2023-10-01	5000.00
502	Shaily	Female	28	Malaria	2023-10-05	25000.00
503	Het	Male	72	Fever	2023-10-08	12000.00
504	Yashvi	Female	45	Flu	2023-10-10	30000.00
505	Karan	Male	15	Injured Leg	2023-10-12	15000.00
506	Dhruhi	Female	30	Fever	2023-10-15	8000.00

Query 6:	Display all patient records.
Answer:	SELECT * FROM patient;

OUTPUT:

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
501	Himanshu	Male	65	Fever	2023-10-01	5000.00
502	Shaily	Female	28	Malaria	2023-10-05	25000.00
503	Het	Male	72	Fever	2023-10-08	12000.00
504	Yashvi	Female	45	Flu	2023-10-10	30000.00
505	Karan	Male	15	Injured Leg	2023-10-12	15000.00
506	Dhruhi	Female	30	Fever	2023-10-15	8000.00

Query 7:	Display patient_name and disease only.
Answer:	SELECT patient_name, disease FROM patient;

OUTPUT:

patient_name	disease
Himanshu	Fever
Shaily	Malaria
Het	Fever
Yashvi	Flu
Karan	Injured Leg
Dhruhi	Fever

Query 8:	Display patients suffering from 'Fever'.
Answer:	SELECT * FROM patient WHERE disease = 'Fever';

OUTPUT:

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
501	Himanshu	Male	65	Fever	2023-10-01	5000.00
503	Het	Male	72	Fever	2023-10-08	12000.00
506	Dhruhi	Female	30	Fever	2023-10-15	8000.00

Query 9:	Display patients whose bill_amount is greater than 20000.
Answer:	SELECT * FROM patient WHERE bill_amount > 20000;

OUTPUT:

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
502	Shaily	Female	28	Malaria	2023-10-05	25000.00
504	Yashvi	Female	45	Flu	2023-10-10	30000.00

Query 10:	Display patients ordered by bill_amount (descending).
Answer:	SELECT * FROM patient ORDER BY bill_amount DESC;

OUTPUT:

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
504	Yashvi	Female	45	Flu	2023-10-10	30000.00
502	Shaily	Female	28	Malaria	2023-10-05	25000.00
505	Karan	Male	15	Injured Leg	2023-10-12	15000.00
503	Het	Male	72	Fever	2023-10-08	12000.00
506	Dhruhi	Female	30	Fever	2023-10-15	8000.00
501	Himanshu	Male	65	Fever	2023-10-01	5000.00

Query 11:	Display patients whose age is above 60.
Answer:	SELECT * FROM patient WHERE age > 60;

OUTPUT;

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
501	Himanshu	Male	65	Fever	2023-10-01	5000.00
503	Het	Male	72	Fever	2023-10-08	12000.00

Query 12:	Update bill_amount of a patient using patient_id.
Answer:	UPDATE patient SET bill_amount = 5500.00 WHERE patient_id = 501;

OUTPUT:

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
501	Himanshu	Male	65	Fever	2023-10-01	5500.00
502	Shaily	Female	28	Malaria	2023-10-05	25000.00
503	Het	Male	72	Fever	2023-10-08	12000.00
504	Yashvi	Female	45	Flu	2023-10-10	30000.00
505	Karan	Male	15	Injured Leg	2023-10-12	15000.00
506	Dhruhi	Female	30	Fever	2023-10-15	8000.00

Query 13:	Increase bill_amount of all patients by 1000.
Answer:	UPDATE patient SET bill_amount = bill_amount + 1000;

OUTPUT:

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
501	Himanshu	Male	65	Fever	2023-10-01	6500.00
502	Shaily	Female	28	Malaria	2023-10-05	26000.00
503	Het	Male	72	Fever	2023-10-08	13000.00
504	Yashvi	Female	45	Flu	2023-10-10	31000.00
505	Karan	Male	15	Injured Leg	2023-10-12	16000.00
506	Dhruhi	Female	30	Fever	2023-10-15	9000.00

Query 14:	Delete a patient record using patient_id.
Answer:	DELETE FROM patient WHERE patient_id = 505;

OUTPUT:

patient_id	patient_name	gender	age	disease	admission_date	bill_amount
501	Himanshu	Male	65	Fever	2023-10-01	6500.00
502	Shaily	Female	28	Malaria	2023-10-05	26000.00
503	Het	Male	72	Fever	2023-10-08	13000.00
504	Yashvi	Female	45	Flu	2023-10-10	31000.00
506	Dhruhi	Female	30	Fever	2023-10-15	9000.00

Query 15:	Count total patients.
Answer:	SELECT COUNT(*) AS TotalPatient FROM patient;

OUTPUT:

TotalPatient
5

Query 16:	Find the highest bill_amount.

Answer:	SELECT MAX(bill_amount) AS HighestBillAmount FROM patient;

OUTPUT:

```
+--------------------+
| HighestBillAmount |
+--------------------+
|           31000.00 |
+--------------------+
```

Query 17:	Find average bill_amount.
Answer:	SELECT AVG(bill_amount) AS AverageBillAmount FROM patient;

OUTPUT:

```
+--------------------+
| AverageBillAmount |
+--------------------+
|       17100.000000 |
+--------------------+
```

DBMS [Assignment – 6]

Table Name: course_enroll		
Sr. No	**Field Name**	**Data Type**
1	enroll_id	INTEGER
2	student_name	VARCHAR(100)
3	course_name	VARCHAR(100)
4	duration_months	INTEGER
5	fees	DECIMAL(10,2)
6	enrollment_date	DATE

Query 1:	Create a database named academy_db.
Answer:	CREATE DATABASE academy_db; USE academy_db;
Query 2:	Create the above table and Set enroll_id as Primary Key.
Answer:	CREATE TABLE course_enroll (enroll_id INT PRIMARY KEY, student_name VARCHAR(100), course_name VARCHAR(100), duration_months INT, fees DECIMAL(10, 2), enrollment_date DATE);
Query 3:	Display the structure of the course_enroll table.
Answer:	DESC course_enroll;
OUTPUT:	

```
+-----------------+---------------+------+-----+---------+-------+
| Field           | Type          | Null | Key | Default | Extra |
+-----------------+---------------+------+-----+---------+-------+
| enroll_id       | int           | NO   | PRI | NULL    |       |
| student_name    | varchar(100)  | YES  |     | NULL    |       |
| course_name     | varchar(100)  | YES  |     | NULL    |       |
| duration_months | int           | YES  |     | NULL    |       |
| fees            | decimal(10,2) | YES  |     | NULL    |       |
| enrollment_date | date          | YES  |     | NULL    |       |
+-----------------+---------------+------+-----+---------+-------+
```

Query 4:	Insert at least 5 enrollment records.
Answer:	INSERT INTO course_enroll VALUES (1001, 'Alice', 'Python', 3, 15000.00, '2023-09-01'), (1002, 'Bob', 'Data Science', 8, 45000.00, '2023-09-05'), (1003, 'Charlie', 'Web Dev', 6, 25000.00, '2023-09-10'), (1004, 'David', 'Python', 4, 18000.00, '2023-09-15'), (1005, 'Eve', 'Cyber Security', 12, 50000.00, '2023-09-20') ;

OUTPUT:

```
+-----------+--------------+----------------+-----------------+----------+-----------------+
| enroll_id | student_name | course_name    | duration_months | fees     | enrollment_date |
+-----------+--------------+----------------+-----------------+----------+-----------------+
|      1001 | Alice        | Python         |               3 | 15000.00 | 2023-09-01      |
|      1002 | Bob          | Data Science   |               8 | 45000.00 | 2023-09-05      |
|      1003 | Charlie      | Web Dev        |               6 | 25000.00 | 2023-09-10      |
|      1004 | David        | Python         |               4 | 18000.00 | 2023-09-15      |
|      1005 | Eve          | Cyber Security |              12 | 50000.00 | 2023-09-20      |
+-----------+--------------+----------------+-----------------+----------+-----------------+
```

Query 5:	Insert one record using column names.
Answer:	INSERT INTO course_enroll (enroll_id, student_name, course_name, duration_months, fees, enrollment_date) VALUES (1006, 'Frank', 'Java', 5, 20000.00, '2023-09-25');

OUTPUT:

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1001	Alice	Python	3	15000.00	2023-09-01
1002	Bob	Data Science	8	45000.00	2023-09-05
1003	Charlie	Web Dev	6	25000.00	2023-09-10
1004	David	Python	4	18000.00	2023-09-15
1005	Eve	Cyber Security	12	50000.00	2023-09-20
1006	Frank	Java	5	20000.00	2023-09-25

Query 6:	Display all enrollment records.
Answer:	SELECT * FROM course_enroll;

OUTPUT:

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1001	Alice	Python	3	15000.00	2023-09-01
1002	Bob	Data Science	8	45000.00	2023-09-05
1003	Charlie	Web Dev	6	25000.00	2023-09-10
1004	David	Python	4	18000.00	2023-09-15
1005	Eve	Cyber Security	12	50000.00	2023-09-20
1006	Frank	Java	5	20000.00	2023-09-25

Query 7:	Display student_name and course_name only.
Answer:	SELECT student_name, course_name FROM course_enroll;

OUTPUT:

student_name	course_name
Alice	Python
Bob	Data Science
Charlie	Web Dev
David	Python
Eve	Cyber Security
Frank	Java

Query 8:	Display students enrolled in 'Python' course.
Answer:	SELECT * FROM course_enroll WHERE course_name = 'Python';

OUTPUT:

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1001	Alice	Python	3	15000.00	2023-09-01
1004	David	Python	4	18000.00	2023-09-15

Query 9:	Display enrollments with fees greater than 30000.
Answer:	SELECT * FROM course_enroll WHERE fees > 30000;

OUTPUT:

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1002	Bob	Data Science	8	45000.00	2023-09-05
1005	Eve	Cyber Security	12	50000.00	2023-09-20

Query 10:	Display enrollments ordered by fees (ascending).
Answer:	SELECT * FROM course_enroll ORDER BY fees;

OUTPUT:

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1001	Alice	Python	3	15000.00	2023-09-01
1004	David	Python	4	18000.00	2023-09-15
1006	Frank	Java	5	20000.00	2023-09-25
1003	Charlie	Web Dev	6	25000.00	2023-09-10
1002	Bob	Data Science	8	45000.00	2023-09-05
1005	Eve	Cyber Security	12	50000.00	2023-09-20

Query 11:	Display courses with duration more than 6 months.
Answer:	SELECT * FROM course_enroll WHERE duration_months > 6;

OUTPUT;

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1002	Bob	Data Science	8	45000.00	2023-09-05
1005	Eve	Cyber Security	12	50000.00	2023-09-20

Query 12:	Update fees of a course using enroll_id.
Answer:	UPDATE course_enroll SET fees = 16000.00 WHERE enroll_id = 1001;

OUTPUT:

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1001	Alice	Python	3	16000.00	2023-09-01
1002	Bob	Data Science	8	45000.00	2023-09-05
1003	Charlie	Web Dev	6	25000.00	2023-09-10
1004	David	Python	4	18000.00	2023-09-15
1005	Eve	Cyber Security	12	50000.00	2023-09-20
1006	Frank	Java	5	20000.00	2023-09-25

Query 13:	Increase fees of all courses by 10%.
Answer:	UPDATE course_enroll SET fees = fees * 1.10;

OUTPUT:

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1001	Alice	Python	3	17600.00	2023-09-01
1002	Bob	Data Science	8	49500.00	2023-09-05
1003	Charlie	Web Dev	6	27500.00	2023-09-10
1004	David	Python	4	19800.00	2023-09-15
1005	Eve	Cyber Security	12	55000.00	2023-09-20
1006	Frank	Java	5	22000.00	2023-09-25

Query 14:	Delete an enrollment using enroll_id.
Answer:	DELETE FROM course_enroll WHERE enroll_id = 1006;

OUTPUT:

enroll_id	student_name	course_name	duration_months	fees	enrollment_date
1001	Alice	Python	3	17600.00	2023-09-01
1002	Bob	Data Science	8	49500.00	2023-09-05
1003	Charlie	Web Dev	6	27500.00	2023-09-10
1004	David	Python	4	19800.00	2023-09-15
1005	Eve	Cyber Security	12	55000.00	2023-09-20

Query 15:	Count total enrollments.
Answer:	SELECT COUNT(*) AS TotalEnrollments FROM course_enroll;

OUTPUT:

TotalEnrollments
5

Query 16:	Find maximum course fees.
Answer:	SELECT MAX(fees) AS MaximumCourse FROM course_enroll;

OUTPUT:

MaximumCourse
55000.00

Query 17:	Find average course fees.
Answer:	SELECT AVG(fees) AS AverageFees FROM course_enroll;

OUTPUT:

```
+--------------+
| AverageFees  |
+--------------+
| 33880.000000 |
+--------------+
```

DBMS [Assignment – 7]

Table Name: vehicle		
Sr. No	**Field Name**	**Data Type**
1	vehical_id	INTEGER
2	owner_name	VARCHAR(100)
3	vehicle_type	VARCHAR(50)
4	registration_no	VARCHAR(20)
5	registration_date	DATE
6	fuel_type	VARCHAR(50)

Query 1:	Create a database named transport_db.
Answer:	CREATE DATABASE transport_db; USE transport_db;
Query 2:	Create the above table and Set vehicle_id as Primary Key.
Answer:	CREATE TABLE vehicle (vehicle_id INT PRIMARY KEY, owner_name VARCHAR(100), vehicle_type VARCHAR(50), registration_no VARCHAR(20), registration_date DATE, fuel_type VARCHAR(20));
Query 3:	Display the structure of the vehicle table.
Answer:	DESC vehicle;
OUTPUT:	

```
+------------------+--------------+------+-----+---------+-------+
| Field            | Type         | Null | Key | Default | Extra |
+------------------+--------------+------+-----+---------+-------+
| vehicle_id       | int          | NO   | PRI | NULL    |       |
| owner_name       | varchar(100) | YES  |     | NULL    |       |
| vehicle_type     | varchar(50)  | YES  |     | NULL    |       |
| registration_no  | varchar(20)  | YES  |     | NULL    |       |
| registration_date| date         | YES  |     | NULL    |       |
| fuel_type        | varchar(20)  | YES  |     | NULL    |       |
+------------------+--------------+------+-----+---------+-------+
```

Query 4:	Insert at least 5 vehicle records.
Answer:	INSERT INTO vehicle VALUES (1, 'Dhruv Sharma', 'Sedan', 'GJ-01-AB-1234', '2021-05-10', 'Petrol'), (2, 'Hemang Desai', 'SUV', 'GJ-05-CD-5678', '2023-01-15', 'Diesel'), (3, 'Disha Patel', 'Hatchback', 'GJ-03-EF-9012', '2022-11-20', 'Petrol'), (4, 'Gurnoor Brar', 'SUV', 'MH-01-GH-3456', '2024-03-05', 'Electric'), (5, 'Akshy Khana', 'Sedan', 'DL-02-IJ-7890', '2023-08-12', 'CNG') ;

OUTPUT:

```
+------------+--------------+--------------+-----------------+-------------------+-----------+
| vehicle_id | owner_name   | vehicle_type | registration_no | registration_date | fuel_type |
+------------+--------------+--------------+-----------------+-------------------+-----------+
|          1 | Dhruv Sharma | Sedan        | GJ-01-AB-1234   | 2021-05-10        | Petrol    |
|          2 | Hemang Desai | SUV          | GJ-05-CD-5678   | 2023-01-15        | Diesel    |
|          3 | Disha Patel  | Hatchback    | GJ-03-EF-9012   | 2022-11-20        | Petrol    |
|          4 | Gurnoor Brar | SUV          | MH-01-GH-3456   | 2024-03-05        | Electric  |
|          5 | Akshy Khana  | Sedan        | DL-02-IJ-7890   | 2023-08-12        | CNG       |
+------------+--------------+--------------+-----------------+-------------------+-----------+
```

Query 5:	Insert one record using column names.
Answer:	INSERT INTO vehicle (vehicle_id, owner_name, vehicle_type, registration_no, registration_date, fuel_type) VALUES (6, 'Ankur Shah','Hatchback', 'GJ-06-KL-1122', '2024-01-10', 'Petrol');

OUTPUT:

vehicle_id	owner_name	vehicle_type	registration_no	registration_date	fuel_type
1	Dhruv Sharma	Sedan	GJ-01-AB-1234	2021-05-10	Petrol
2	Hemang Desai	SUV	GJ-05-CD-5678	2023-01-15	Diesel
3	Disha Patel	Hatchback	GJ-03-EF-9012	2022-11-20	Petrol
4	Gurnoor Brar	SUV	MH-01-GH-3456	2024-03-05	Electric
5	Akshy Khana	Sedan	DL-02-IJ-7890	2023-08-12	CNG
6	Ankur Shah	Hatchback	GJ-06-KL-1122	2024-01-10	Petrol

Query 6:	Display all vehicle records.
Answer:	SELECT * FROM vehicle;

OUTPUT:

vehicle_id	owner_name	vehicle_type	registration_no	registration_date	fuel_type
1	Dhruv Sharma	Sedan	GJ-01-AB-1234	2021-05-10	Petrol
2	Hemang Desai	SUV	GJ-05-CD-5678	2023-01-15	Diesel
3	Disha Patel	Hatchback	GJ-03-EF-9012	2022-11-20	Petrol
4	Gurnoor Brar	SUV	MH-01-GH-3456	2024-03-05	Electric
5	Akshy Khana	Sedan	DL-02-IJ-7890	2023-08-12	CNG
6	Ankur Shah	Hatchback	GJ-06-KL-1122	2024-01-10	Petrol

Query 7:	Display owner_name and vehicle_type only.
Answer:	SELECT owner_name, vehicle_type FROM vehicle;

OUTPUT:

owner_name	vehicle_type
Dhruv Sharma	Sedan
Hemang Desai	SUV
Disha Patel	Hatchback
Gurnoor Brar	SUV
Akshy Khana	Sedan
Ankur Shah	Hatchback

Query 8:	Display vehicles using 'Petrol' fuel.
Answer:	SELECT * FROM vehicle WHERE fuel_type = 'Petrol';

OUTPUT:

vehicle_id	owner_name	vehicle_type	registration_no	registration_date	fuel_type
1	Dhruv Sharma	Sedan	GJ-01-AB-1234	2021-05-10	Petrol
3	Disha Patel	Hatchback	GJ-03-EF-9012	2022-11-20	Petrol
6	Ankur Shah	Hatchback	GJ-06-KL-1122	2024-01-10	Petrol

Query 9:	Display vehicles registered after the year 2022.
Answer:	SELECT * FROM vehicle WHERE registration_date > '2022-12-31';

OUTPUT:

vehicle_id	owner_name	vehicle_type	registration_no	registration_date	fuel_type
2	Hemang Desai	SUV	GJ-05-CD-5678	2023-01-15	Diesel
4	Gurnoor Brar	SUV	MH-01-GH-3456	2024-03-05	Electric
5	Akshy Khana	Sedan	DL-02-IJ-7890	2023-08-12	CNG
6	Ankur Shah	Hatchback	GJ-06-KL-1122	2024-01-10	Petrol

Query 10:	Display vehicles ordered by registration_date.
Answer:	SELECT * FROM vehicle ORDER BY registration_date;

OUTPUT:

vehicle_id	owner_name	vehicle_type	registration_no	registration_date	fuel_type
1	Dhruv Sharma	Sedan	GJ-01-AB-1234	2021-05-10	Petrol
3	Disha Patel	Hatchback	GJ-03-EF-9012	2022-11-20	Petrol
2	Hemang Desai	SUV	GJ-05-CD-5678	2023-01-15	Diesel
5	Akshy Khana	Sedan	DL-02-IJ-7890	2023-08-12	CNG
6	Ankur Shah	Hatchback	GJ-06-KL-1122	2024-01-10	Petrol
4	Gurnoor Brar	SUV	MH-01-GH-3456	2024-03-05	Electric

Query 11:	Update fuel_type of a vehicle using vehicle_id.
Answer:	UPDATE vehicle SET fuel_type = 'Electric' WHERE vehicle_id = 1;

OUTPUT;

vehicle_id	owner_name	vehicle_type	registration_no	registration_date	fuel_type
1	Dhruv Sharma	Sedan	GJ-01-AB-1234	2021-05-10	Electric
2	Hemang Desai	SUV	GJ-05-CD-5678	2023-01-15	Diesel
3	Disha Patel	Hatchback	GJ-03-EF-9012	2022-11-20	Petrol
4	Gurnoor Brar	SUV	MH-01-GH-3456	2024-03-05	Electric
5	Akshy Khana	Sedan	DL-02-IJ-7890	2023-08-12	CNG
6	Ankur Shah	Hatchback	GJ-06-KL-1122	2024-01-10	Petrol

Query 12:	Change vehicle_type of an owner.
Answer:	UPDATE vehicle SET vehicle_type = 'SUV' WHERE owner_name = 'Kevin Patel';

OUTPUT:

vehicle_id	owner_name	vehicle_type	registration_no	registration_date	fuel_type
1	Dhruv Sharma	Sedan	GJ-01-AB-1234	2021-05-10	Electric
2	Hemang Desai	SUV	GJ-05-CD-5678	2023-01-15	Diesel
3	Disha Patel	SUV	GJ-03-EF-9012	2022-11-20	Petrol
4	Gurnoor Brar	SUV	MH-01-GH-3456	2024-03-05	Electric
5	Akshy Khana	Sedan	DL-02-IJ-7890	2023-08-12	CNG
6	Ankur Shah	Hatchback	GJ-06-KL-1122	2024-01-10	Petrol

Query 13:	Delete a vehicle record using vehicle_id.
Answer:	DELETE FROM vehicle WHERE vehicle_id = 5;

OUTPUT:

vehicle_id	owner_name	vehicle_type	registration_no	registration_date	fuel_type
1	Dhruv Sharma	Sedan	GJ-01-AB-1234	2021-05-10	Electric
2	Hemang Desai	SUV	GJ-05-CD-5678	2023-01-15	Diesel
3	Disha Patel	SUV	GJ-03-EF-9012	2022-11-20	Petrol
4	Gurnoor Brar	SUV	MH-01-GH-3456	2024-03-05	Electric
6	Ankur Shah	Hatchback	GJ-06-KL-1122	2024-01-10	Petrol

Query 14:	Count total vehicles.
Answer:	SELECT COUNT(*) AS TotalVehicle FROM vehicle;

OUTPUT:

TotalVehicle
5

Query 15:	Find total number of Petrol vehicles.
Answer:	SELECT COUNT(*) AS TotalPetrolVechile FROM vehicle WHERE fuel_type = 'Petrol';

OUTPUT:

TotalPetrolVechile
2

DBMS [Assignment – 8]

Table Name: electricity_bill		
Sr. No	**Field Name**	**Data Type**
1	consumer_id	INTEGER
2	consumer_name	VARCHAR(100)
3	city	VARCHAR(50)
4	units_consumed	INTEGER
5	bill_month	VARCHAR(20)
6	amount	DECIMAL(10,2)

Query 1:	Create a database named electricity_db.
Answer:	CREATE DATABASE electricity_db; USE electricity_db;
Query 2:	Create the above table and Set consumer_id as Primary Key.
Answer:	CREATE TABLE electricity_bill (consumer_id INT PRIMARY KEY, consumer_name VARCHAR(100), city VARCHAR(50), units_consumed INT, bill_month VARCHAR(20), amount DECIMAL(10, 2));
Query 3:	Display the structure of the electricity_bill table.
Answer:	DESC electricity_bill;
OUTPUT:	

```
+----------------+---------------+------+-----+---------+-------+
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| consumer_id    | int           | NO   | PRI | NULL    |       |
| consumer_name  | varchar(100)  | YES  |     | NULL    |       |
| city           | varchar(50)   | YES  |     | NULL    |       |
| units_consumed | int           | YES  |     | NULL    |       |
| bill_month     | varchar(20)   | YES  |     | NULL    |       |
| amount         | decimal(10,2) | YES  |     | NULL    |       |
+----------------+---------------+------+-----+---------+-------+
```

Query 4:	Insert at least 5 consumer records.
Answer:	INSERT INTO electricity_bill VALUES (101, 'Ankur Shah', 'Surat', 350, 'January', 2100.00), (102, 'Disha Patel', 'Ahmedabad', 150, 'January', 900.00), (103, 'Chetan Jariwala', 'Surat', 420, 'January', 2800.00), (104, 'Monak Patel', 'Rajkot', 280, 'February', 1540.00), (105, 'Virat Kohli', 'Surat', 120, 'February', 720.00) ;

OUTPUT:

```
+-------------+-----------------+-----------+----------------+------------+---------+
| consumer_id | consumer_name   | city      | units_consumed | bill_month | amount  |
+-------------+-----------------+-----------+----------------+------------+---------+
|         101 | Ankur Shah      | Surat     |            350 | January    | 2100.00 |
|         102 | Disha Patel     | Ahmedabad |            150 | January    |  900.00 |
|         103 | Chetan Jariwala | Surat     |            420 | January    | 2800.00 |
|         104 | Monak Patel     | Rajkot    |            280 | February   | 1540.00 |
|         105 | Virat Kohli     | Surat     |            120 | February   |  720.00 |
+-------------+-----------------+-----------+----------------+------------+---------+
```

Query 5:	Insert one record using column names.
Answer:	INSERT INTO electricity_bill (consumer_id, consumer_name, city, units_consumed, bill_month, amount) VALUES (106, 'Amir Khan', 'Vadodara', 310, 'February', 1860.00);

OUTPUT:

consumer_id	consumer_name	city	units_consumed	bill_month	amount
101	Ankur Shah	Surat	350	January	2100.00
102	Disha Patel	Ahmedabad	150	January	900.00
103	Chetan Jariwala	Surat	420	January	2800.00
104	Monak Patel	Rajkot	280	February	1540.00
105	Virat Kohli	Surat	120	February	720.00
106	Amir Khan	Vadodara	310	February	1860.00

Query 6:	Display all electricity bill records.
Answer:	SELECT * FROM electricity_bill;

OUTPUT:

consumer_id	consumer_name	city	units_consumed	bill_month	amount
101	Ankur Shah	Surat	350	January	2100.00
102	Disha Patel	Ahmedabad	150	January	900.00
103	Chetan Jariwala	Surat	420	January	2800.00
104	Monak Patel	Rajkot	280	February	1540.00
105	Virat Kohli	Surat	120	February	720.00
106	Amir Khan	Vadodara	310	February	1860.00

Query 7:	Display consumer_name and amount only.
Answer:	SELECT consumer_name, amount FROM electricity_bill;

OUTPUT:

consumer_name	amount
Ankur Shah	2100.00
Disha Patel	900.00
Chetan Jariwala	2800.00
Monak Patel	1540.00
Virat Kohli	720.00
Amir Khan	1860.00

Query 8:	Display consumers from 'Surat' city.
Answer:	SELECT * FROM electricity_bill WHERE city = 'Surat';

OUTPUT:

consumer_id	consumer_name	city	units_consumed	bill_month	amount
101	Ankur Shah	Surat	350	January	2100.00
103	Chetan Jariwala	Surat	420	January	2800.00
105	Virat Kohli	Surat	120	February	720.00

Query 9:	Display consumers whose units_consumed is greater than 300.
Answer:	SELECT * FROM electricity_bill WHERE units_consumed > 300;

OUTPUT:

consumer_id	consumer_name	city	units_consumed	bill_month	amount
101	Ankur Shah	Surat	350	January	2100.00
103	Chetan Jariwala	Surat	420	January	2800.00
106	Amir Khan	Vadodara	310	February	1860.00

Query 10:	Display bills ordered by amount (descending).
Answer:	SELECT * FROM electricity_bill ORDER BY amount DESC;

OUTPUT:

consumer_id	consumer_name	city	units_consumed	bill_month	amount
103	Chetan Jariwala	Surat	420	January	2800.00
101	Ankur Shah	Surat	350	January	2100.00
106	Amir Khan	Vadodara	310	February	1860.00
104	Monak Patel	Rajkot	280	February	1540.00
102	Disha Patel	Ahmedabad	150	January	900.00
105	Virat Kohli	Surat	120	February	720.00

Query 11:	Update amount of a consumer using consumer_id.
Answer:	UPDATE electricity_bill SET amount = 2200.00 WHERE consumer_id = 101;

OUTPUT;

consumer_id	consumer_name	city	units_consumed	bill_month	amount
101	Ankur Shah	Surat	350	January	2200.00
102	Disha Patel	Ahmedabad	150	January	900.00
103	Chetan Jariwala	Surat	420	January	2800.00
104	Monak Patel	Rajkot	280	February	1540.00
105	Virat Kohli	Surat	120	February	720.00
106	Amir Khan	Vadodara	310	February	1860.00

Query 12:	Increase the amount of all consumers by 8%.
Answer:	UPDATE electricity_bill SET amount = amount * 1.08;

OUTPUT:

consumer_id	consumer_name	city	units_consumed	bill_month	amount
101	Ankur Shah	Surat	350	January	2376.00
102	Disha Patel	Ahmedabad	150	January	972.00
103	Chetan Jariwala	Surat	420	January	3024.00
104	Monak Patel	Rajkot	280	February	1663.20
105	Virat Kohli	Surat	120	February	777.60
106	Amir Khan	Vadodara	310	February	2008.80

Query 13:	Delete a consumer record using consumer_id.
Answer:	DELETE FROM electricity_bill WHERE consumer_id = 105;

OUTPUT:

consumer_id	consumer_name	city	units_consumed	bill_month	amount
101	Ankur Shah	Surat	350	January	2376.00
102	Disha Patel	Ahmedabad	150	January	972.00
103	Chetan Jariwala	Surat	420	January	3024.00
104	Monak Patel	Rajkot	280	February	1663.20
106	Amir Khan	Vadodara	310	February	2008.80

Query 14:	Count total consumers.
Answer:	SELECT COUNT(*) AS TotalConsumer FROM electricity_bill;

OUTPUT:

TotalConsumer
5

Query 15:	Find the maximum electricity bill amount.
Answer:	SELECT MAX(amount) AS MaximumBillAmount FROM electricity_bill;

OUTPUT:

MaximumBillAmount
3024.00

DBMS [Assignment – 9]

Table Name: booking		
Sr. No	**Field Name**	**Data Type**
1	booking_id	INTEGER
2	guest_name	VARCHAR(100)
3	room_type	VARCHAR(50)
4	city	VARCHAR(50)
5	check_in_date	DATE
6	check_out_date	DATE
7	total_amount	DECIMAL(10,2)

Query 1:	Create a database named hotel_db.
Answer:	CREATE DATABASE hotel_db; USE hotel_db;
Query 2:	Create the above table and Set booking_id as Primary Key.
Answer:	CREATE TABLE booking (booking_id INT PRIMARY KEY, guest_name VARCHAR(100), room_type VARCHAR(50), city VARCHAR(50), check_in_date DATE, check_out_date DATE, total_amount DECIMAL(10, 2));
Query 3:	Display the structure of the booking table.
Answer:	DESC booking;
OUTPUT:	

```
+----------------+---------------+------+-----+---------+-------+
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| booking_id     | int           | NO   | PRI | NULL    |       |
| guest_name     | varchar(100)  | YES  |     | NULL    |       |
| room_type      | varchar(50)   | YES  |     | NULL    |       |
| city           | varchar(50)   | YES  |     | NULL    |       |
| check_in_date  | date          | YES  |     | NULL    |       |
| check_out_date | date          | YES  |     | NULL    |       |
| total_amount   | decimal(10,2) | YES  |     | NULL    |       |
+----------------+---------------+------+-----+---------+-------+
```

Query 4:	Insert at least 5 booking records.
Answer:	INSERT INTO booking VALUES (201, 'Raj Jani', 'Deluxe', 'Surat', '2023-11-01', '2023-11-05', 18000.00), (202, 'Dev Mehta', 'Standard', 'Mumbai', '2023-11-02', '2023-11-03', 5000.00), (203, 'Kaarn Rana', 'Deluxe', 'Surat', '2023-11-10', '2023-11-12', 12000.00), (204, 'Siya Patel', 'Suite', 'Delhi', '2023-11-15', '2023-11-20', 45000.00), (205, 'Om Anjirwala', 'Standard', 'Pune', '2023-11-18', '2023-11-19', 4500.00) ;

OUTPUT:

```
+------------+--------------+-----------+--------+---------------+----------------+--------------+
| booking_id | guest_name   | room_type | city   | check_in_date | check_out_date | total_amount |
+------------+--------------+-----------+--------+---------------+----------------+--------------+
|        201 | Raj Jani     | Deluxe    | Surat  | 2023-11-01    | 2023-11-05     |     18000.00 |
|        202 | Dev Mehta    | Standard  | Mumbai | 2023-11-02    | 2023-11-03     |      5000.00 |
|        203 | Kaarn Rana   | Deluxe    | Surat  | 2023-11-10    | 2023-11-12     |     12000.00 |
|        204 | Siya Patel   | Suite     | Delhi  | 2023-11-15    | 2023-11-20     |     45000.00 |
|        205 | Om Anjirwala | Standard  | Pune   | 2023-11-18    | 2023-11-19     |      4500.00 |
+------------+--------------+-----------+--------+---------------+----------------+--------------+
```

Query 5:	Insert one booking record using column names.
Answer:	INSERT INTO booking (booking_id, guest_name, room_type, city, check_in_date, Check_out_date, total_amount) VALUES (206, 'Ankur Shah', 'Deluxe', 'Surat', '2023-11-22', '2023-11-25', 16500.00);

OUTPUT:

booking_id	guest_name	room_type	city	check_in_date	check_out_date	total_amount
201	Raj Jani	Deluxe	Surat	2023-11-01	2023-11-05	18000.00
202	Dev Mehta	Standard	Mumbai	2023-11-02	2023-11-03	5000.00
203	Kaarn Rana	Deluxe	Surat	2023-11-10	2023-11-12	12000.00
204	Siya Patel	Suite	Delhi	2023-11-15	2023-11-20	45000.00
205	Om Anjirwala	Standard	Pune	2023-11-18	2023-11-19	4500.00
206	Ankur Shah	Deluxe	Surat	2023-11-22	2023-11-25	16500.00

Query 6:	Display all booking records.
Answer:	SELECT * FROM booking;

OUTPUT:

booking_id	guest_name	room_type	city	check_in_date	check_out_date	total_amount
201	Raj Jani	Deluxe	Surat	2023-11-01	2023-11-05	18000.00
202	Dev Mehta	Standard	Mumbai	2023-11-02	2023-11-03	5000.00
203	Kaarn Rana	Deluxe	Surat	2023-11-10	2023-11-12	12000.00
204	Siya Patel	Suite	Delhi	2023-11-15	2023-11-20	45000.00
205	Om Anjirwala	Standard	Pune	2023-11-18	2023-11-19	4500.00
206	Ankur Shah	Deluxe	Surat	2023-11-22	2023-11-25	16500.00

Query 7:	Display guest_name and room_type only.
Answer:	SELECT guest_name, room_type FROM booking;

OUTPUT:

guest_name	room_type
Raj Jani	Deluxe
Dev Mehta	Standard
Kaarn Rana	Deluxe
Siya Patel	Suite
Om Anjirwala	Standard
Ankur Shah	Deluxe

Query 8:	Display bookings for 'Deluxe' room type.
Answer:	SELECT * FROM booking WHERE room_type = 'Deluxe';

OUTPUT:

booking_id	guest_name	room_type	city	check_in_date	check_out_date	total_amount
201	Raj Jani	Deluxe	Surat	2023-11-01	2023-11-05	18000.00
203	Kaarn Rana	Deluxe	Surat	2023-11-10	2023-11-12	12000.00
206	Ankur Shah	Deluxe	Surat	2023-11-22	2023-11-25	16500.00

Query 9:	Display bookings with total_amount greater than 15000.
Answer:	SELECT * FROM booking WHERE total_amount > 15000;

OUTPUT:

booking_id	guest_name	room_type	city	check_in_date	check_out_date	total_amount
201	Raj Jani	Deluxe	Surat	2023-11-01	2023-11-05	18000.00
204	Siya Patel	Suite	Delhi	2023-11-15	2023-11-20	45000.00
206	Ankur Shah	Deluxe	Surat	2023-11-22	2023-11-25	16500.00

Query 10:	Display bookings ordered by check_in_date.
Answer:	SELECT * FROM booking ORDER BY check_in_date;

OUTPUT:

booking_id	guest_name	room_type	city	check_in_date	check_out_date	total_amount
201	Raj Jani	Deluxe	Surat	2023-11-01	2023-11-05	18000.00
202	Dev Mehta	Standard	Mumbai	2023-11-02	2023-11-03	5000.00
203	Kaarn Rana	Deluxe	Surat	2023-11-10	2023-11-12	12000.00
204	Siya Patel	Suite	Delhi	2023-11-15	2023-11-20	45000.00
205	Om Anjirwala	Standard	Pune	2023-11-18	2023-11-19	4500.00
206	Ankur Shah	Deluxe	Surat	2023-11-22	2023-11-25	16500.00

Query 11:	Update total_amount of a booking using booking_id.
Answer:	UPDATE booking SET total_amount = 19000.00 WHERE booking_id = 201;

OUTPUT;

booking_id	guest_name	room_type	city	check_in_date	check_out_date	total_amount
201	Raj Jani	Deluxe	Surat	2023-11-01	2023-11-05	19000.00
202	Dev Mehta	Standard	Mumbai	2023-11-02	2023-11-03	5000.00
203	Kaarn Rana	Deluxe	Surat	2023-11-10	2023-11-12	12000.00
204	Siya Patel	Suite	Delhi	2023-11-15	2023-11-20	45000.00
205	Om Anjirwala	Standard	Pune	2023-11-18	2023-11-19	4500.00
206	Ankur Shah	Deluxe	Surat	2023-11-22	2023-11-25	16500.00

Query 12:	Increase total_amount of all bookings by 10%.
Answer:	UPDATE booking SET total_amount = total_amount * 1.10;

OUTPUT:

booking_id	guest_name	room_type	city	check_in_date	check_out_date	total_amount
201	Raj Jani	Deluxe	Surat	2023-11-01	2023-11-05	20900.00
202	Dev Mehta	Standard	Mumbai	2023-11-02	2023-11-03	5500.00
203	Kaarn Rana	Deluxe	Surat	2023-11-10	2023-11-12	13200.00
204	Siya Patel	Suite	Delhi	2023-11-15	2023-11-20	49500.00
205	Om Anjirwala	Standard	Pune	2023-11-18	2023-11-19	4950.00
206	Ankur Shah	Deluxe	Surat	2023-11-22	2023-11-25	18150.00

Query 13:	Delete a booking record using booking_id.
Answer:	DELETE FROM booking WHERE booking_id = 205;

OUTPUT:

booking_id	guest_name	room_type	city	check_in_date	check_out_date	total_amount
201	Raj Jani	Deluxe	Surat	2023-11-01	2023-11-05	20900.00
202	Dev Mehta	Standard	Mumbai	2023-11-02	2023-11-03	5500.00
203	Kaarn Rana	Deluxe	Surat	2023-11-10	2023-11-12	13200.00
204	Siya Patel	Suite	Delhi	2023-11-15	2023-11-20	49500.00
206	Ankur Shah	Deluxe	Surat	2023-11-22	2023-11-25	18150.00

Query 14:	Count total bookings.
Answer:	SELECT COUNT(*) AS TotalBookings FROM booking;

OUTPUT:

TotalBookings
5

Query 15:	Find the highest booking amount.
Answer:	SELECT MAX(total_amount) AS HighestBookingAmount FROM booking;

OUTPUT:

HighestBookingAmount
49500.00

DBMS [Assignment – 10]

Table Name: orders		
Sr. No	**Field Name**	**Data Type**
1	orders_id	INTEGER
2	customer_name	VARCHAR(100)
3	product_name	VARCHAR(100)
4	category	VARCHAR(50)
5	order_date	DATE
6	quantity	INTEGER
7	total_price	DECIMAL(10,2)

Query 1:	Create a database named ecommerce_db.
Answer:	CREATE DATABASE ecommerce_db; USE ecommerce_db;
Query 2:	Create the above table and Set orders_id as Primary Key.
Answer:	CREATE TABLE orders (order_id INT PRIMARY KEY, customer_name VARCHAR(100), product_name VARCHAR(100), category VARCHAR(50), order_date DATE, quantity INT, total_price DECIMAL(10, 2));
Query 3:	Display the structure of the orders table.
Answer:	DESC orders;
OUTPUT:	

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	NULL	
customer_name	varchar(100)	YES		NULL	
product_name	varchar(100)	YES		NULL	
category	varchar(50)	YES		NULL	
order_date	date	YES		NULL	
quantity	int	YES		NULL	
total_price	decimal(10,2)	YES		NULL	

Query 4:	Insert at least 5 order records.
Answer:	INSERT INTO orders VALUES (3001, 'Avinash', 'Monitor', 'Electronics', '2023-12-01', 1, 12000.00), (3002, 'Bijal', 'Printer', 'Electronics', '2023-12-05', 3, 15000.00), (3003, 'Chintan', 'T-Shirt', 'Fashion', '2023-12-10', 2, 1000.00), (3004, 'Dax', 'Mouse', 'Electronics', '2023-12-12', 5, 2500.00), (3005, 'Ketan', 'Jeans', 'Fashion', '2023-12-15', 1, 1800.00) ;

OUTPUT:

order_id	customer_name	product_name	category	order_date	quantity	total_price
3001	Avinash	Monitor	Electronics	2023-12-01	1	12000.00
3002	Bijal	Printer	Electronics	2023-12-05	3	15000.00
3003	Chintan	T-Shirt	Fashion	2023-12-10	2	1000.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2500.00
3005	Ketan	Jeans	Fashion	2023-12-15	1	1800.00

Query 5:	Insert one order record using column names.
Answer:	INSERT INTO orders (order_id, customer_name, product_name, category, order_date, quantity, total_price) VALUES (3006, 'Feroz', 'SSD', 'Electronics', '2023-12-18', 2, 7000.00);

OUTPUT:

order_id	customer_name	product_name	category	order_date	quantity	total_price
3001	Avinash	Monitor	Electronics	2023-12-01	1	12000.00
3002	Bijal	Printer	Electronics	2023-12-05	3	15000.00
3003	Chintan	T-Shirt	Fashion	2023-12-10	2	1000.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2500.00
3005	Ketan	Jeans	Fashion	2023-12-15	1	1800.00
3006	Feroz	SSD	Electronics	2023-12-18	2	7000.00

Query 6:	Display all order records.
Answer:	SELECT * FROM orders;

OUTPUT:

order_id	customer_name	product_name	category	order_date	quantity	total_price
3001	Avinash	Monitor	Electronics	2023-12-01	1	12000.00
3002	Bijal	Printer	Electronics	2023-12-05	3	15000.00
3003	Chintan	T-Shirt	Fashion	2023-12-10	2	1000.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2500.00
3005	Ketan	Jeans	Fashion	2023-12-15	1	1800.00
3006	Feroz	SSD	Electronics	2023-12-18	2	7000.00

Query 7:	Display customer_name and product_name only.
Answer:	SELECT customer_name, product_name FROM orders;

OUTPUT:

customer_name	product_name
Avinash	Monitor
Bijal	Printer
Chintan	T-Shirt
Dax	Mouse
Ketan	Jeans
Feroz	SSD

Query 8:	Display orders from 'Electronics' category.
Answer:	SELECT * FROM orders WHERE category = 'Electronics';

OUTPUT:

order_id	customer_name	product_name	category	order_date	quantity	total_price
3001	Avinash	Monitor	Electronics	2023-12-01	1	12000.00
3002	Bijal	Printer	Electronics	2023-12-05	3	15000.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2500.00
3006	Feroz	SSD	Electronics	2023-12-18	2	7000.00

Query 9:	Display orders with quantities greater than 2.
Answer:	SELECT * FROM orders WHERE quantity > 2;

OUTPUT:

order_id	customer_name	product_name	category	order_date	quantity	total_price
3002	Bijal	Printer	Electronics	2023-12-05	3	15000.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2500.00

Query 10:	Display orders ordered by total_price (descending).
Answer:	SELECT * FROM orders ORDER BY total_price DESC;

OUTPUT:

order_id	customer_name	product_name	category	order_date	quantity	total_price
3002	Bijal	Printer	Electronics	2023-12-05	3	15000.00
3001	Avinash	Monitor	Electronics	2023-12-01	1	12000.00
3006	Feroz	SSD	Electronics	2023-12-18	2	7000.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2500.00
3005	Ketan	Jeans	Fashion	2023-12-15	1	1800.00
3003	Chintan	T-Shirt	Fashion	2023-12-10	2	1000.00

Query 11:	Update quantity of an order using order_id.
Answer:	UPDATE orders SET quantity = 4 WHERE order_id = 3002;

OUTPUT;

order_id	customer_name	product_name	category	order_date	quantity	total_price
3001	Avinash	Monitor	Electronics	2023-12-01	1	12000.00
3002	Bijal	Printer	Electronics	2023-12-05	4	15000.00
3003	Chintan	T-Shirt	Fashion	2023-12-10	2	1000.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2500.00
3005	Ketan	Jeans	Fashion	2023-12-15	1	1800.00
3006	Feroz	SSD	Electronics	2023-12-18	2	7000.00

Query 12:	Increase total_price of all orders by 5%.
Answer:	UPDATE orders SET total_price = total_price * 1.05;

OUTPUT:

order_id	customer_name	product_name	category	order_date	quantity	total_price
3001	Avinash	Monitor	Electronics	2023-12-01	1	12600.00
3002	Bijal	Printer	Electronics	2023-12-05	4	15750.00
3003	Chintan	T-Shirt	Fashion	2023-12-10	2	1050.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2625.00
3005	Ketan	Jeans	Fashion	2023-12-15	1	1890.00
3006	Feroz	SSD	Electronics	2023-12-18	2	7350.00

Query 13:	Delete an order record using order_id.
Answer:	DELETE FROM orders WHERE order_id = 3005;

OUTPUT:

order_id	customer_name	product_name	category	order_date	quantity	total_price
3001	Avinash	Monitor	Electronics	2023-12-01	1	12600.00
3002	Bijal	Printer	Electronics	2023-12-05	4	15750.00
3003	Chintan	T-Shirt	Fashion	2023-12-10	2	1050.00
3004	Dax	Mouse	Electronics	2023-12-12	5	2625.00
3006	Feroz	SSD	Electronics	2023-12-18	2	7350.00

Query 14:	Count total orders.
Answer:	SELECT COUNT(*) AS TotalOrders FROM orders;

OUTPUT:

TotalOrders
5

Query 15:	Find the maximum order price.
Answer:	SELECT MAX(total_price) AS MaximumOrder FROM orders;

OUTPUT:

MaximumOrder
15750.00